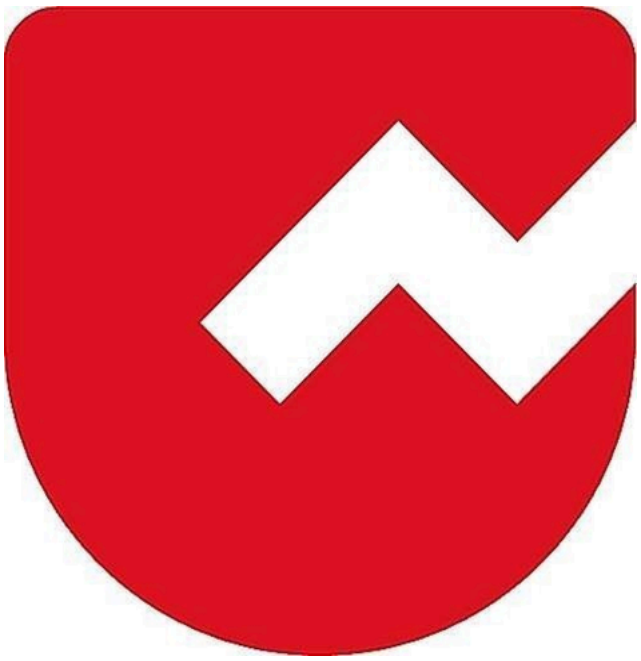
Zoho Schools for Graduate Studies



Notes

String Part - I

1. What is a String ?

In java , a String is an Object that represents a sequence of characters.

2. How is a String literal represented in Java?

In Java, a **String literal** is a sequence of characters enclosed within double quotes (" ").

```
package part1;

public class StringDemo{

 public static void main(String[] args)

        {

            String str = "Hello";

        }

 }
```

String Literal in Python

In Python, a string literal is also represented by enclosing characters in quotes.

Single quotes ' '

Double quotes " "

Triple quotes ''' ''' or """ """ (for multi-line strings).

```
'Hello',"Hello",'''Hello'''
```

3. Which package does String class belongs to ?

In java, the String class belongs to the java.lang.package.

import java.lang.String;

4. What are the ways of creating a String ?

1. **Using String Literal**

   One way of creating a String is by directly assigning a literal to a variable

   String s1 = "Hello";

2. **Using new Keyword**

   The other way is by using the new keyword.

   String s2 = new String("Hello");

```
public class StringDemo{
    public static void main(String[] argos)
    {
String str1 = “Hello”;
String str2 = “Hello”;
String str3 = new String(“Hello”);
}
}
```

5. What is the difference between creating a String using a literal and using the new keyword in java?

```
public class StringDemo{
    public static void main(String[] args)
```

```
  {
  String str1 = “Hello”;
  String str2 = “Hello”; // String constant pool Heap memory
  String str3 = new String(“Hello”);
  System.out.println(str1 + “ “ +str2 + “ “+str3);
  System.out.println(str1==str2); // checks reference equality(memory address)
  System.out.println(str2==str3); // Different memory location so false
  }
}
```

Output:

Hello Hello Hello

true

False

(‘==’) Equality operator checks of similarity of memory references .

```
  String str1 = “Hello”;
  String str2 = “Hello”; // String constant pool
  String str3 = new String(“Hello”);
  System.out.println(str1 + “ “ +str2 + “ “+str3);
  System.out.println(str1==str2);
  System.out.println(str2==str3);
  System.out.println(str1.equals(str2)); // Check the content similarity
  System.out.println(str1.equals(str3));
```

Output:

Hello HelloHello

true

false

true

True

Equality operator VS .Equals () Method:

`==` **(Equality Operator)**	`.equals()` **Method**
Compares **references (memory addresses)**	Compares **contents/values** (if overridden in class)
Checks reference equality only	Same as `==` (Object's default `equals()` also checks reference)
Checks if both references point to the same object in memory (SCP vs Heap matters)	Overridden to compare actual characters (content)
Only if both references point to the exact same object	If the **values inside** objects are same

6. Creating a Multi-Line String

MultiLine String can be enclosed within Triple double quotes closed in java

```
public class StringDemo {

 public static void main(String[] args) {

      String str1 = """

      Hello

      Mndhbdhbdhb ///Perfectly legal in java

      """;

 String str2 = "Hello"; // String constant pool

 String str3 = new String("Hello");

 System.out.println(str1 + " “ +str2 + " "+str3);

 System.out.println(str1==str2);

System.out.println(str2==str3);

System.out.println(str1.equals(str2)); // Check the content similarity

System.out.println(str1.equals(str3));


  }

}
```

Output:

mndhbdhbdhb

Hello Hello

Whenever you used new operator then objects are created becomes a objects created only in Heap Memory.

Which is a preferred approach ?

Use String literals or Use new keyword

The choice depends on the **problem definition**

String are immutable can not modify the string

7. String Methods :

1. **length()** - Return length of string

   System.out.pritnln(str2.length); - compiler error

   String str2 = “Hello”;

   String str4 = “God”;

   System.out.println(str2.length()); // no of characters

   Output:

   5

2. **concat() -** joins two strings together

   String str2 = “Hello”; //String constant pool

   String str4 = “God”;

   str2.concat(str4); // can not modify the original string - output-Hello

   System.out.println(str2);

   //concatenation

   str2 =str2.concat(str4); //output - HelloGod

   Concat () - create new memory location will be created.

3. **SubString()** -

   ```
   public class StringDemo {
      public static void main(String[] args) {
         String str2 ="Hello";
   ```

```
        String str4="God";
        str2=str2.concat(str4);
        System.out.println(str2);
        System.out.println(str2.substring(4,6));  //it will stop till 5th index
    }
}
```

Output:

HelloGod

oG

System.out.println(str2.substring(4,4)); //used same index

Output:

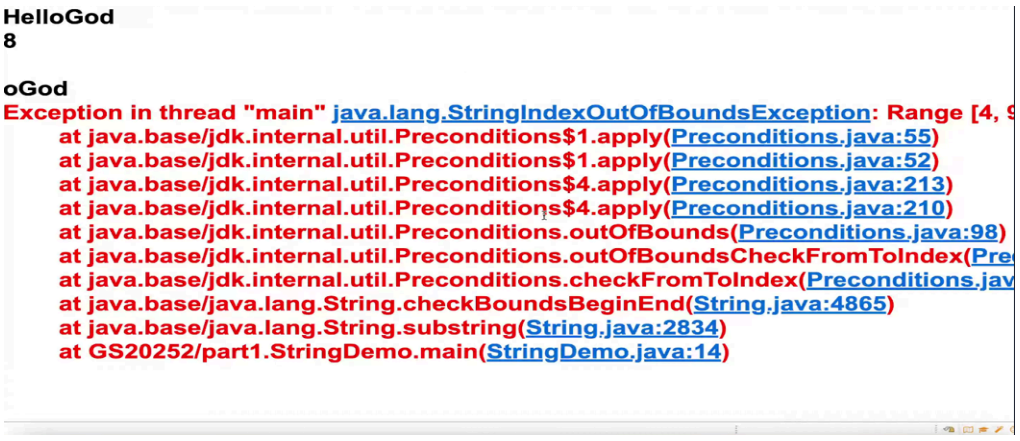
Empty String

System.out.println(str2.substring(4,8)); // Substring end of last index

Output:

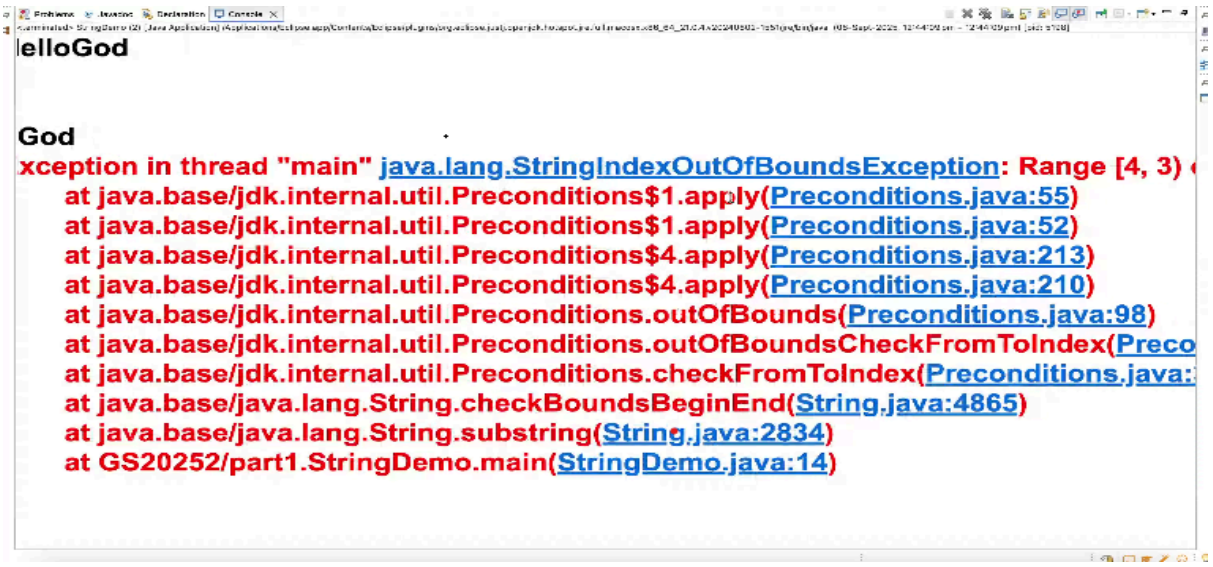
8

System.out.println(str2.substring(4,9)); // Runtime error



String index outOf bound Exception

System.out.println(str2.substring(4,3)); //Runtime Exceptions



System.out.println(str2.substring(4)); // From 4th index till end of the string

Output:

oGod

4. **startsWith()-** Checks whether a string begins with a given prefix.Returns true/false.

   System.out.println(str2.startsWith("lo")); // false

5. **EndsWith()-** Checks whether a string ends with a given suffix.Returns true/false.

   System.out.println(str4.endsWith("od")); // true

6. **charAt() -** Returns the character at a specific position(index)in a string

   System.out.println("str2.charAt(4): " + str2.charAt(4)); // 'o'

7. **indexOf()-** It accepts the string . four Overloaded method in indexOf().

   System.out.println(str2.indexOf('o'));

   Output:

   4

   System.out.println(str2.indexOf('f')); // -1 the character not available

8. **stripLeading()**-Removes all leading(starting) whitespace characters from a string.Similar to trim(), but trim() removes both leading and trailing spaces , while stripLeading() removes only from the start.

   System.out.println(str2.stripLeading());

   Output:

   HelloGod

9. **stripTrailing() -** removes **trailing spaces** only.

   Output:

   HelloGod

10. **trim() -** Removes leading (starting) and trailing (ending) spaces from a string.

11. **getBytes() -** It converts a string into a equivalent byte array based on the character encoding.

    Character take two bytes.

    Output:

    [B@17550481

12. **Arrays.toString.getBytes()**-Print the byte array values in readable way.

    System.out.println(Arrays.toString(str2.getBytes());

    Space ascii value - 32

    Output:

    32,32,32,72,101,108,108,111,100,32,32,32] //Equivalent Integer.

13. **toCharArray()** -Converts a string into character array(char[]).Each character in the string becomes one element in the array.

    System.out.println(Arrays.toString(str2.toCharArray());

    Output:

    [, , , H,e,l,l,o,G,o,d, , ,]

```
public class StringDemo {

    public static void main(String[] args) {

        String str2 ="Hello";

        String str4="God";

       str2=str2.concat(str4);

       System.out.println(str2);//HelloGod

       System.out.println(str2.length());

       System.out.println(str2.substring(4,4));

       System.out.println(str2.substring(4,8));

       System.out.println(str2.substring(4));

       System.out.println(str2.indexOf("f"));

       System.out.println(str2.stripLeading());

       System.out.println(str2.stripTrailing());

       System.out.println(str2.trim());

       System.out.println(Arrays.toString(str2.getBytes()));

       System.out.println(Arrays.toString(str2.toCharArray()));

       System.out.println(str2.startsWith("he"));

       System.out.println(str2.endsWith(" "));

    }

}
```

Output:

HelloGod

8

oGod

oGod

-1

HelloGod

HelloGod

HelloGod

[72, 101, 108, 108, 111, 71, 111, 100]

[H, e, l, l, o, G, o, d]

false

false